情報処理学会論文誌　Vol.59　No.1　1–12 (Jan. 2018)

大規模言語モデルを用いた適応ルールの自動構築と最適化

石水 優成[1]　許 経略[2]　山内 拓人[3]　蔡 金雨[3]　伊庭 斉志[2]　李 家隆[1,3,a)]　鄭 顕志[1]

受付日 xxxx年0月xx日, 採録日 xxxx年0月xx日

概要：ルールベースの自己適応システムでは，外部環境や内部状態に応じて事前に定義された適応ルールに基づき，運用中にシステムの動作や構成が変更される．この手法の利点は，実行時のオンラインプランニングが不要であるため迅速な適応が可能であり，さらに運用仕様（通常はソースコード）が高い可読性を有することである．しかし，適切な状態空間を用いた適応ルールを構築するには，深いドメイン知識を持つ専門家によるの状態空間設計の試行錯誤が必要であり，少ない人的努力で効率的なルール設計を実現することが課題となっている．そこで，本論文では，大規模言語モデル（LLM）の意味的推論能力を活用した適応ルールの自動構築および最適化手法を提案する．提案では，(1) 運用履歴から適応ルールにおける問題箇所を特定し，(2) ルールの改善案を分析，の２つのプロセスを繰り返すことでルールの構築と最適化を実現する．サーバー負荷分散の事例での実験を通じて，本提案手法は，人間が手動設計したルールと同等の性能を達成しつつ，設計コストを大幅に削減できることが確認された．

キーワード：適応ルール, 大規模言語モデル, 状態空間設計, 最適化

Automatic Generation and Optimization of Adaptation Rules using Large Language Models

Yusei Ishimizu[1]　Jinglue Xu[2]　Takuto Yamauchi[3]　Jinyu Cai[3]　Hitoshi Iba[2]　Jialong Li[1,3,a)]
Kenji Tei[1]

Received: xx xx, xxxx, Accepted: xx xx, xxxx

Abstract: In rule-based self-adaptive systems, the system's behavior or configuration is dynamically adjusted during runtime based on predefined adaptation rules that respond to external environmental changes or internal states. This approach offers the advantage of enabling rapid adaptation without requiring online planning at runtime, while also providing highly readable operational specifications, typically expressed as source code. However, crafting adaptation rules with an appropriate state space requires extensive trial and error in designing the state space by experts with deep domain knowledge, posing a significant challenge to achieving efficient rule design with minimal human effort. To address this, we propose LLM-ARG, a novel approach that leverages the semantic reasoning capabilities of large language models (LLMs) for the automated construction and optimization of adaptation rules. LLM-ARGiterates through two key processes: (1) identifying issues in the adaptation rules from execution logs, and (2) analyzing and proposing improvements to the rule, thereby achieving rule construction and optimization. Through experiments using a server load balancing case study, we demonstrate that LLM-ARG achieves performance comparable to human-designed rules while significantly reducing design costs.

Keywords: Adaptation Rule, Large Language Models, State Space Design, Optimization

1. はじめに

自己適応システム（SAS: Self-Adaptive Systems）とは，

[1] 東京科学大学, 2-12-1 Ookayama, Meguro-ku, Tokyo, Japan
[2] 東京大学, 7-3-1 Hongo, Bunkyo-ku, Tokyo, Japan
[3] 早稲田大学, 1-104 Totsukamachi, Shinjuku-ku, Tokyo, Japan
[a)] Corresponding_Author:lijialong@fuji.waseda.jp

© 2018 Information Processing Society of Japan

運用中に外部環境の変化や内部状態の変動に応じて，自律的に動作や構成を動的に変更する能力を持つシステムである．自己適応システムにおける適応アプローチは，設計目的や運用環境によって多様であるが，中でもルールベースの適応 [1][13] は広く採用される代表的な手法である．適応ルールは「IF (状態) THEN (アクション)」で定式化することが可能である．サーバー群の負荷分散を管理するロードーバランサーを例に挙げると，状態は「ユーザリクエストに対する応答時間」,「サーバーの利用率」, アクションは「サーバーを削除する」,「サーバーを追加する」などが考えられる．

ルールベースのアプローチでは，特定の条件が満たされると動作や構成が変更される仕様が，事前に定義された適応ルールとして記述される．この手法の利点は，ルールが直感的かつ簡潔に設計可能である点である．さらに，事前に定義されたルールに基づくため，システムは実行時に複雑な推論や計画を必要とせず，環境変化に迅速に対応する．この特性は，リアルタイム適応が求められるシステムで特に有用である．また，動作仕様が透明性と解釈可能性を備えた形式（例：ソースコード）であるため，透明性や変更が容易である点でも優れている [17].

適応ルールの構築は，システムの状態とアクションの組み合わせの中から，システムの目標（例えば，応答時間やコストの削減）を最大化するルールを見つけるプロセスである．これは，状態空間とアクション空間を探索し，最適な政策（policy）を見つけ出す最適化問題として捉えられる．これまでの研究では，進化計算や強化学習などの最適化アルゴリズムを用いて，状態やアクションに関する数値（例えば，応答時間の閾値や追加するサーバーの数）を最適化する技術が提案されてきた [22], [23].

しかし，これらの自動化手法は数値的な最適化に特化しており，変数が持つ意味や関係性を十分に考慮していない．本論文でいう「セマンティックな設計・最適化」とは，単なる数値パラメータ調整に留まらず，変数が持つ意味やドメイン知識に基づき，生の観測データからシステムの目標達成に有効な，より抽象度の高い状態変数を新たに導き出し，定義する行為を指す[*1]．たとえば，応答時間を状態として考える場合，素朴に個々のユーザーリクエストに対する応答時間の組み合わせ（例：<0.3s, 0.4s, 0.7s>）を状態として扱うことは可能であるが，状態空間が膨大になり，ルールの設計や保守のしやすさが損なわれる．一方,「平均応答時間」を状態として扱う場合，状態空間のサイズを抑えつつシステムの全体的な傾向を捉えられる利点がある．しかし，平均応答時間を用いる場合でも,「何秒間のデータを平均化すべきか」「全リクエストを対象にするか，20%の最悪リクエストを対象にするか」といった数多くの状態空間設計の選択肢が考えられる．これらのような，システム目標や運用特性，例えば，サーバ起動には 120 秒を要するといったようなドメイン固有の特性に即した抽象化を行うことが「セマンティックな設計」にあたる．セマンティックな状態空間の設計と最適化は，ドメイン知見を持つ分野専門家が実際のドメインの特徴に合わせて試行錯誤して設計する必要がある．その結果，人的負担が増大し，開発コストが上昇する課題が生じる．

そこで本論文では，大規模言語モデル（Large Language Model, 以下 LLM と略記）のセマンティックな推論能力を活用し，適応ルールの構築と最適化手法 LLM-ARG（LLM-driven Adaptation Rule Generator）を提案する．具体的には,「既存の運用履歴からルールの問題箇所特定」および「問題箇所の修正と改善」の 2 段階を用いて，ルールにおける状態空間設計の改良を繰り返すことで，ルールの構築と最適化を実現する．具体的には，本論文の貢献点は以下の通りである．

- LLM を用いた適応ルールの自動構築と最適化フレームワークの提案
- サーバー負荷分散の例題を用いて，性能実験とユーザ実験の両方を含んだ包括的な評価実験

本研究の初期アイデアと実装は先行論文 [11] で報告されているが，本稿では以下の点で重要な改良と詳細な検証を行っている: (1) 入力プロンプトの構造化: 先行研究の自然文ベースのプロンプトが持つ曖昧さを解消するため，システムの変数やドメイン知識を厳密な JSON 形式で構造化し，推論の安定性と保守性を高めた．(2) ルール改良とコード生成の分離: 単一プロンプトによる End-to-End 方式では不完全なコードが生成されがちだったため，プロセスを「改善方針の生成」と「コード生成」の 2 段階に分離し，各タスクの単純化を通じて生成コードの品質を向上させた．(3) 包括的な評価の実施: 提案手法の有効性を多角的に検証するため，性能実験に加えてユーザ実験を含む，より包括的な評価を実施した．

本論文の構成は以下の通りである．第 2 章では，例題を用いて本研究が取り扱う問題を定義する．第 3 章では，提案手法 LLM-ARG の詳細を述べる．第 4 章では，ユーザ評価実験を通じて LLM-ARG の有効性を検証し，LLM を用いた適応ルール構築における試行錯誤の合理性を議論する．第 5 章では関連研究を概観し，第 6 章で本論文の結論をまとめる．

2. 問題定義と例題

2.1 適応ルールとその最適化

ルールベースの自己適応システムでは，環境の変化に応じて事前に定義された適応ルールに基づいて振る舞う．適応ルールは,「IF (状態) THEN (アクション)」の形式で定式化される．「状態」はシステム内の変数の組み合わせに

*1 モデリングの視点では modelling dimensions とも呼ばれる [5]

© 2018 Information Processing Society of Japan

よって定義され，特定の条件が満たされると「アクション」が実行される．「アクション」はシステム内の可変パラメータを制御する方法を指定し，適応のための具体的な操作を意味する．

本研究では，システムの適応目標である効用関数を最大化することを，以下で定義される最適化問題として扱う．

$$\max_{\mathbf{a}\in\mathcal{A}} \sum_t U(\mathbf{s}, \mathcal{S}, \mathbf{a}, t)$$

ここでは $\mathcal{S}$ はすべての可能な状態の集合であり，$\mathbf{s} \in \mathcal{S}$ はシステムの現在の状態である． $\mathbf{a} \in \mathcal{A}$ は，システムが取るアクションであり，すべてのアクションの集合 $\mathcal{A}$ に属する． $U(\mathbf{s}, \mathcal{S}, \mathbf{a}, t)$ は効用関数であり，ある時間 t において，状態 $\mathbf{s} \in \mathcal{S}$ でアクション $\mathbf{a}$ を達成した性能を評価する．特に注意すべきなのは，本研究では $\mathcal{S}$ も最適化の対象である点となる．

2.2　例題：Znn.com

自己適応システムの研究で最も代表的なサーバ負荷分散「Znn.com」を例題として説明する [6].

ベース状態空間：システムのベースとなる状態を $\mathcal{S}$ として，リクエスト到着率 a，稼働中のサーバー数 s，dimmer 値 d，各リクエストの応答時間 r_i などの変数が含まれる．これらの変数は時間とともに変化し，システムが適応すべき状況を反映する．適応ルールの構築にあたり，これらのベース状態を活用するほか，ベース状態に基づいて新しい状態を定義することも求められている．

アクション空間：システムが取る行動はアクションの集合 $\mathcal{A}$ に属する $\mathbf{a}$ として，以下の 2 つの具体的なアクションが用意されている：

- サーバーの追加/削減（$addServer/reduceServer$）サーバープール内のサーバー数を増減させ，リクエスト処理能力を調整する．サーバー数は 0 から事前に定められた上限 $MaxServers$ の範囲で制御される．また，サーバー起動，これらのアクションの実行からサーバー稼働開始までの遅延を 120 秒とする．
- dimmer 値の設定（$setDimmer(dimmer)$）$dimmer$ は 0 から 1 の値であり，オプショナルコンテンツを含むレスポンスの確率を調整する．

効用関数：多くの先行研究で使われていた効用関数（Utility Function）を利用する．具体的には，サーバーの運用コスト，応答時間，オプショナルコンテンツ（広告など）の含有率といった指標を統合し，以下のように定義される：

$$U = \sum_t UTILITY_t$$

$$UTILITY_t = \begin{cases} U_R + U_C & (r \le T \text{ and } U_R \ge U_R^*) \\ U_R & (r \le T \text{ and } U_R < U_R^*) \\ R_o \cdot \min(0, a - \kappa) & (r > T) \end{cases}$$

ここで，以下の変数が定義される：

- $U_R = a \cdot (d \cdot R_o + (1-d) \cdot R_b)$：収益に関連する項
- $U_C = c \cdot (s^* - s)$：サーバー運用コストに関連する項
- T：応答時間の閾値
- r：時間 t における平均応答時間
- $U_R^* = a \cdot R_o$
- d：$dimmer$ の値
- R_o, R_b：オプショナル収益，基本収益
- c：サーバー 1 台あたりの稼働コスト
- s^*：サーバーの最大数
- κ：$maxThroughput = s^* \cdot maxServiceRate$
- $maxServiceRate$：1 台のサーバーが処理できる最大リクエスト到着数/秒

状態空間のセマンティックな設計と最適化：効用関数を最大化するためには，状態空間を正確かつ意味的に反映するよう設計する必要がある[*2]．例えば，Znn.com のシステムでは，各リクエストの応答時間を観察・取得できた場合，その情報を適応ルールの状態空間としてこれらの情報をそのまま利用するのが難しいと考えられる．そのため，複数のリクエストの応答時間を集約して「平均応答時間」という状態を新たに定義することが考えられる．その際，「平均応答時間」の定義にはいくつかの意思決定が伴う．具体的には，直近の短期間の平均を取るか，長期間の平均を取るか（短時間のピーク負荷に対応するのか，それとも長期間の傾向に対応するのか），また，何秒間のデータを平均化するのが合理的か（サーバーの起動時間との兼ね合いも考慮する必要がある）といった点が考えられる．このように，状態空間の設計と最適化は，適応目標，ドメイン知識や実運用環境などの情報に基づいてをセマンティックに行われることが求められる．

3.　提案手法: LLM-ARG

本研究では，LLM の知識と推論能力を活用し，適応ルールの生成と改善を行う LLM-ARG を提案する．本手法は，1 章で述べた「既存の運用履歴からルールの問題箇所特定」と「問題箇所の修正と改善」という 2 つの主要段階を反復的に実行する．図 1 に示す通り，このプロセスは Knowledge Base，Monitor，Problem Identifier，Rule Refiner，Rule Generator の 5 つのコンポーネントによって具体化される．具体的には，段階①は Problem Identifier が対応し，段階②は Rule Refiner（改善方針の策定）と Rule Generator（コードへの具体化）の 2 つのコンポーネントによって

*2　ソフトウェア工学のコンテキストでは，クラス内のメンバ変数の設計に該当．

© 2018 Information Processing Society of Japan

実現される．Monitor は段階①のためのデータを提供し，Knowledge Base は全プロセスを通じて必要なドメイン知識や履歴データを供給する役割を担う．本章の残りの部分では，Znn.com の例題を用いて提案手法の各コンポーネントの役割と実際の出力例を解説する．なお，本章ではプロンプトの詳細な説明を省略し，プロンプト構造に関する説明は付録に記載する．

3.1 Knowledge Base

LLM の推論能力を最大限に活用するには，包括的かつ構造化されたコンテキストを提供することが不可欠である．これを実現するため，Knowledge base は複数の情報を保持するよう設計されている．具体的には，現在の適応ルールのほか，リスト 1 に示す例のように，以下の 4 種類の情報が蓄積される．まず，(1) 対象システムの動作ロジックやドメイン知識などの情報である．これらの情報は，Znn.com のシミュレータを紹介するツール論文 [16] から GPT-4 を用いて自動的に抜粋したものであり，たとえばサーバー負荷分散の基本動作や制約条件が含まれる．一例としては，「サーバー数追加した場合，サーバの稼働（起動時間）まで 120 秒間必要とする (line 3)」が挙げられる．これにより，LLM はシステムの挙動を事前に把握し，適切なルール設計の基礎となる．次に，(2) 適応目標を定義し，対象システムが達成すべき目的を明確化する情報である．ここでは，多くの先行研究で利用された節 2.2 の効用関数の最大化を適応目標とする (line 6)．さらに，(3) 観測可能な変数の性質 (lines 8-16) や制御可能な変数の制御方法 (lines 17-25) を詳細に記述する情報である．例えば，観測可能な変数には稼働中のサーバー数や応答時間，制御可能な変数には dimmer 値などが挙げられる．さらに，これらの変数の型，範囲，ソースコードとの対応関係（コード生成のためのグラウンディング用）も含まれる．ただし，ルール生成ではこれらの変数の全てを使用するとは限らず，状況に応じて一部を選択したり，新たな変数を定義したりする場合もある．最後に，(4) 実際の運用履歴である (lines 27-36)．これは，リクエスト到着率，サーバー稼働数や効用の時系列データが含まれている．

```
1  {
2   "DomainKnowledge": {
3    "StartDelay": "the delay for starting the server is
         120 seconds",
4                        ...
5   },
6   "AdaptationGoal": "...",
7   "VariableInfo": {
8    "ObservableVariables": {
9     "ActiveServer": {
10     "Description": "number of active servers",
11     "Type": "Integer",
12     "Range": "1 - MaxServers",
13     "Code Impl": "pModel->getActiveServers()",
14    },
15                        ...
16   },
17   "ControllableVariables": {
18    "dimmer": {
19     "Description": "probability of a response
           including the optional content",
20     "Type": "double",
21     "Range": "0-1",
22     "Code Impl": "pMacroTactic->addTactic(new
           SetDimmerTactic(new_dimmer_value))",
23    },
24                        ...
25   },
26  },
27  "ExecutionHistory": {
28   "timestep data": {
29    "Time": 10: {
30     "avgResponseTime": 0.0457,
31     "Servers": 2,
32     "Utility": 22.0,
33                        ...
34    },
35   },
36  }
37 }
```

Listing 1: LLM へのプロンプトの一部として，Knowledge Base が提供する情報の例

3.2 Problem Identifier

Problem Identifier では，運用履歴における問題箇所（例：効用値が平均を下回る期間）を特定し，その問題を引き起こした適応ルールの課題およびソースコード上の対応箇所を明らかにする．具体的には，以下の 2 つの手順で分析を行う．まず，ステップ 1 として，変数の値の時系列データを含む運用履歴から，改善が必要な箇所を特定する．リスト 2 に，LLM の出力例を示す．この例では，応答時間が閾値を大幅に下回り，サーバー数が過剰である可能性が高い期間を特定している．

```
1  ...
2  At Time 960 seconds:
3  - Utility: 33 (this is positive but reflects the
       compromised performance seen in previous time
       slots)
4  - avgResponseTime: 0.06679 (well below threshold)
5  - Servers: 3
6  - ActiveServers: 3
7  - Dimmer: 1
8
9  Cause for Compromised Utility: Despite the good
       response time, with three active servers, it
       might be feasible to reduce the server count
       while maintaining good performance. Additionally
       , the dimmer at max suggests that optional
       content is at its peak.
10 ...
```

Listing 2: 運用履歴における改善箇所特定の例

ステップ 2 では，ステップ 1 で特定した運用履歴の問題箇所の原因を詳細に分析する．適応目標に対して選定した

© 2018 Information Processing Society of Japan

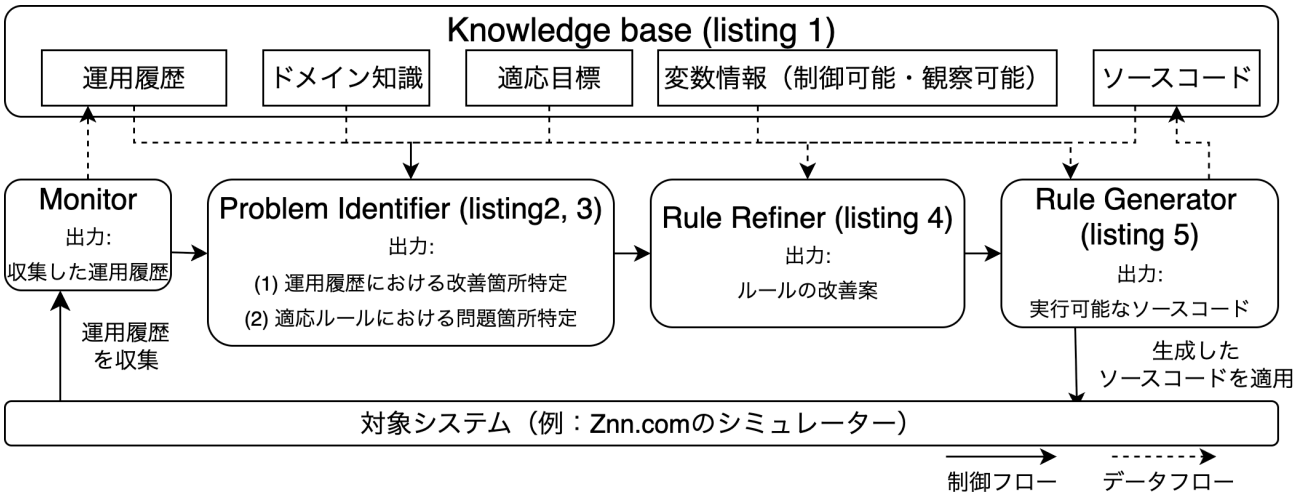


図 1: LLM-ARG の概要図

箇所が改善の余地を生んだ理由を明らかにし，適応ルールの設計における課題を特定する．具体的には，適応ルールを反映したソースコードのどの部分が問題を引き起こしているかを示す．以下に，LLM による原因分析の出力例を示す．リスト 3 では，過去のサーバー利用率だけでの意思決定が不十分だと指摘されている．

```
...
Insufficient Use of Historical Data: Although historical averages are calculated, they are not fully integrated into the decision-making process. For instance, when low average response times are recorded, the logic does not explore the possibility of reducing server numbers more aggressively:

    if (avgHistoricalUtilization < 0.6) {
        pMacroTactic->addTactic(new RemoveServerTactic);
    }
...
```

Listing 3: 適応ルールにおける問題箇所特定の例

3.3 Rule Refiner

Rule Refiner では，Problem Identifer で特定した問題箇所に基づき，適応ルールの改善案を提案する．リスト 4 の出力例では，ステップ 2 で指摘された問題を解消するため，過去のサーバー利用率と応答時間を組み合わせてサーバ数の削減を判断する改善案が示されている．

```
...
Enhanced Reactive Logic: Implement a more nuanced decision-making structure that incorporates both real-time metrics and historical averages to determine when to add or remove servers. For example, when historical utilization consistently trends below a certain threshold, initiate server removal if the average response time is within acceptable limits.
...
```

Listing 4: 適応ルール改善案検討の例

3.4 Rule Generator

Rule Generator，Problem Identifer で特定した問題箇所と Rule Refiner で検討した改善案に基づき，適応ルールの改善を行う．リスト 5 の出力例では，過去のサーバー利用率と応答時間を組み合わせてサーバ数を削減する適応ルールを示す．

```
...
if (avgResponseTime > RT_THRESHOLD || avgHistoricalResponseTime > RT_THRESHOLD) {
    ... // サーバ数増加に関するルールについて省略
} else {
    // Reduce servers if utilization and response times are consistently low
    if (!isServerBooting && currentServers > 1 && avgHistoricalUtilization < 0.5) {
        pMacroTactic->addTactic(new RemoveServerTactic);
    }
    // Increase dimmer to serve more optional content
    dimmer = min(1.0, dimmer + 0.1);
}
...
```

Listing 5: 適応ルール改善案検討の例

4. 評価

4.1 研究課題

本章では，以下の 3 つの研究課題を用いて LLM-ARG の有効性と実用性を検証する．

研究課題 1 (ルール最適化の性能)　LLM-ARG による適応ルールの構築と最適化は可能か？また，従来の人間による手動ルール設計と比較して，LLM-ARG の自動ルール設計による効用はどの程度であるか？

研究課題 2 (ルール構築のコスト)　LLM-ARG の時間的

© 2018 Information Processing Society of Japan

コストと金銭的コストはどの程度であるか？

研究課題 3 (状態空間最適化の合理性)　LLM-ARG による適応ルールの最適化において，LLM が既存ルールの課題を適切に特定し，合理的な改善案を分析しているか？

4.2　実験設定

実験では，章 2 で紹介したサーバー負荷分散の例題を用いて評価を行う. 具体的には，5 つの異なるリクエスト到着率の変動シナリオを設定し，それぞれの実験を実施する. シナリオ 1（`clarknet-http-105m-l70`[18]，図 2 に示す）とシナリオ 2（`worldcup_day53-r0-105m-l70`[16]）は，異なる実ウェブサイトの実履歴データである. 一方，シナリオ 3 とシナリオ 4 は，シナリオ 1 とシナリオ 2 の変動をそれぞれ逆にしたものであり，シナリオ 5 はシナリオ 2 とシナリオ 4 の変化の激しい変化部分のみを連結したシナリオである. 実験では，多くのトークンが消費されるため，実験コストを考慮して GPT-4o mini を利用する. ただし，生成したソースコードの実行可能性を確保するため，コード生成に関わる Rule Generator のみ GPT-4o を用いる.

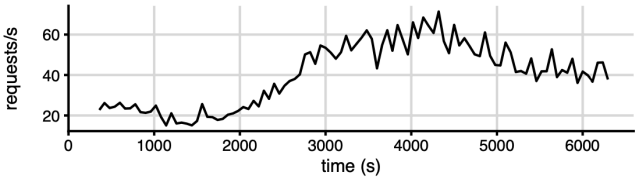


図 2: シナリオ 1 におけるリクエスト到着率の変動

研究課題 1（性能）の評価実験: 適応ルール生成 1 回を「イテレーション」と定義し，1 セットの実験は 10 イテレーションを行う．LLM-ARG の有効性を確認するため，各シナリオで 5 セット（計 50 イテレーション）を実施し，各イテレーションの効用を測定する．コードバグや結果取得不能の場合は ERROR（値なし）として記録し，統計から除外する．さらに，人間との比較としてユーザ実験を行う．具体的には，Znn.com をある程度熟知[*3]する被験者 4 名（情報学分野，博士後期課程 1 名，博士前期課程 3 名）に，シナリオ 1 で 1 セットを実施した．ここでは，被験者実験はシナリオ 1 かつ 1 セットのみ行うのは，複数シナリオによる知識蓄積の影響を排除するためである．

研究課題 2（コスト）の評価実験: 金銭的コストについて，研究課題 1 の実験内で，LLM-ARG の 1 セットの LLM のトークン数・API 使用料金を測定する．時間的なコストについて，LLM-ARG の 1 セットの平均時間，および被験者が設計に使う時間を計測する．

研究課題 3（合理性）の評価実験: 状態空間設計の合理性評価について正解となる ground truth がないため，被験者に，LLM-ARG の状態空間の設計の試行錯誤が合理的かを主観評価してもらう．具体的には，研究課題 1 のユーザ実験実施後，被験者が行ったシナリオ 1 について，被験者それぞれ異なるセットについて LLM の問題分析・改善案生成の合理性を 5 段階評価（1: とても悪い，2: 悪い，3: 普通，4: 良い，5: とても良い）で主観的に判断してもらう．なお，ここでの「合理性」の評価は，LLM が提案した改善プロセス（問題特定から改善案の策定まで）の論理的な妥当性を対象としており，生成された最終的なソースコードの可読性そのものは評価項目に含んでいない．

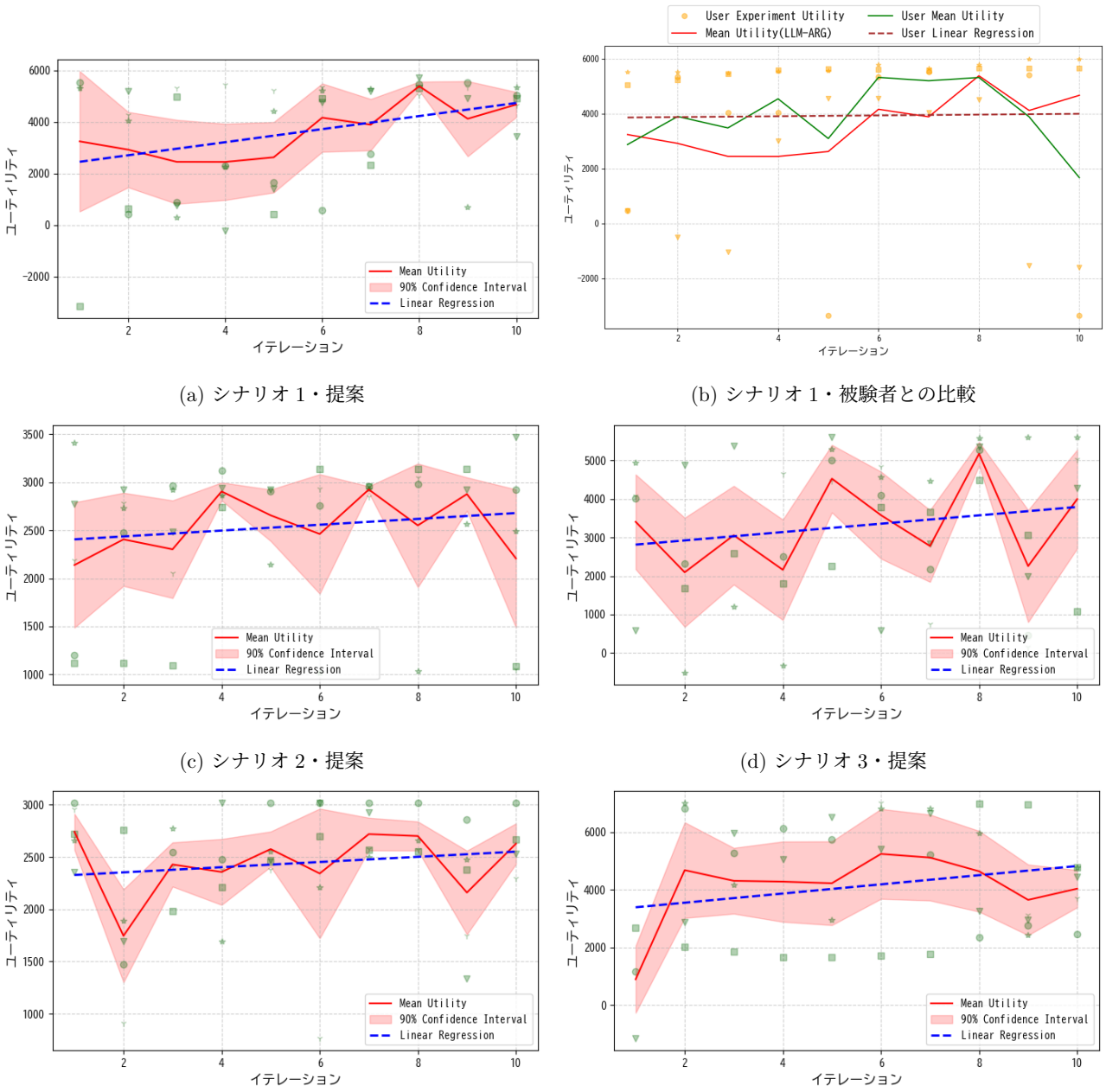
4.3　実験結果

研究課題 1(性能): 研究課題 1 の実験結果は図 3 に示す．横軸はイテレーション，縦軸は効用である．緑色のプロットは試行ごとに形状の異なるマーカーで示した個別結果を表し，赤色の折れ線はその平均値の推移を，青色の点線は回帰直線を表す．LLM-ARG では，シナリオ 1〜5 全てでイテレーションの増加に伴う効用の向上が確認された．回帰直線の傾きはシナリオ 1〜5 でそれぞれ 253.2，30.4，108.8，24.8，159.3 である．特に，シナリオ 3,4,5 は前述の通り，より複雑なシナリオになるように現実のデータを加工したものであるが，これらのシナリオにおいても，イテレーションの増加に伴う効用の向上が観察された．ただし，LLM-ARG では，前のイテレーションでのルールが性能が良かったとしても，次のイテレーションでルールを大きく変更し性能が急激に劣化するケースが多くあった．性能の最も良い適応ルールについて，LLM-ARG の最良値が 5698 であり，被験者 4 名が設計した最良ルールの効用はそれぞれ 5663，5531，4556，5992 である．また，図 3(b) において，形状の異なる黄色のマークは各被験者 4 名の各イテレーションにおける効用（個別値）を示し，茶色の折れ線はその平均値の推移を示す．この図では，被験者個別の効用（黄色のマーク）には試行錯誤による変動が見られるものの，後述するように，これらは開発者による意図的な仮説検証や実装上の試行錯誤の結果である．

研究課題 2（コスト）：時間的なコストについて，LLM-ARG は 1 回のイテレーションを平均 57 秒で完了し，被験者の平均時間（3 分 12 秒）と比較して約 3.3 倍の時間短縮を実現する. 特にイテレーション 1 では，LLM-ARG が 51 秒，被験者の平均が 10 分 10 秒であり，LLM-ARG が約 12 倍の効率を示す. この差は主に，被験者がイテレーション 1 で初期条件の設計に多くの時間を要したことに起因する. しかし，イテレーションが進むにつれて，被験者の処理時間は大幅に短縮される. イテレーション 6 以降では平均約 1 分 26 秒となり，LLM-ARG との差が縮小する. この傾向は，インレーションが進むにつれて，修正箇所が徐々に局所化した結果だと考えられる．

*3 Znn. com を実際に研究の評価例題として扱う，または Znn. com を扱う関連論文を多数熟読した経験がある

© 2018 Information Processing Society of Japan



(a) シナリオ 1・提案

(b) シナリオ 1・被験者との比較

(c) シナリオ 2・提案

(d) シナリオ 3・提案

(e) シナリオ 4・提案

(f) シナリオ 5・提案

図 3: 研究課題 1 の実験結果

また，金銭的なコストについては，各モデルごとの消費トークン数および API 費用を表 1 に示す．ここで，Input Token (Uncached) は LLM に新規で入力されたプロンプトや履歴文脈に含まれるトークン数，Input Token (Cached) は一度キャッシュに格納された履歴文脈を再利用する際に参照されるトークン数，Output Token は LLM が応答として生成したトークン数をそれぞれ指す．料金は，それぞれのトークン数に対してモデル提供元が定める単価を乗じて算出した．1 セットの合計コストは約 0.9 ドル（税抜で約 130 円）である．標準的なソフトウェアエンジニアの時給を 3,000 円と想定した比較にした場合，平均 30 分間を要する人手設計 (1,500 円) と比べて金銭的にも優位であることがわかる．

研究課題 3（合理性）：10 イテレーション分の分析を 4 人に評価させた結果，「すごく悪い」が 1 個，「悪い」が 5 個，「普通」が 10 個，「良い」が 21 個，「すごく良い」が 3 個となった．「LLM が合理的に状態空間の設計を試行錯誤をしているか」の評価平均は 3.5 であり，一定の肯定的な評価を得ている．個々のスコアにばらつきはあるが，LLM の試行錯誤は平均で 3 以上の評価を受けており，被験者がこれを有意義な分析と捉えたことを示す．

4.4　議論と限界

研究課題 1 の結果から，LLM-ARG はイテレーション 1

© 2018 Information Processing Society of Japan

コンポーネント	モデル名	Input Token (Cached)	Input Token (Uncached)	Output Token	Cost (USD)
Problem Identifier,Rule Refiner	gpt-4o-mini	470,528	171,473	18,102	0.08
Rule Generator	gpt-4o	227,328	154,215	14,968	0.81

表 1: 1 セットの実験の平均消費トークン数と API 代

から高い効用を達成し，シナリオ 1〜5 の全てで反復的な改善傾向が確認される．また，人間の手動設計との比較では，LLM-ARG の最良値が 4 人中 3 人について人間を上回った．研究課題 2 では，時間的コストが人間の 3 倍以上低く，金銭的コストが 10 倍以上低いことが確認される．研究課題 3 では，被験者評価の平均が 3.5 であり，LLM の状態空間設計の試行錯誤と改善案が一定の合理性を有することが示される．上記の結果から，LLM を用いた適応ルールの自動構築と最適化の有効性と実用性が確認された．

しかし，限界として，LLM の大胆な設計変更がもたらす安定性の問題が挙げられる．図 3(b) からわかるように，人間の手動設計（茶色の点線）は効用が比較的安定しているのに対し，LLM-ARG による自動設計ではイテレーション間で効用が急激に変動するケースが多く観察される．

この傾向は，人間と LLM の双方のコード履歴全体[*4]を確認した結果からも裏付けられる．具体的には，人間はルールの変更箇所を局所的に留める傾向があり，変更が良い結果に繋がればその構造を維持し，逆に悪い結果になれば一つ前の良好なバージョンに戻す（例えばアクションを実行する条件の閾値のみを調整するなど）といった，安定性を重視した探索を行うことが多く観察される．一方，LLM は直前のイテレーションで良好な結果が得られていたとしても，提示された改善点に対してルール全体の構造を大胆に再構成する傾向があり，これが性能の大きな変動要因となっている．この点は，研究課題 3 の合理性評価においても確認された．具体的には，被験者評価に「悪い」や「とても悪い」が含まれた主な理由として，LLM が提案する設計改良が過度に大胆であり，人間の直観と乖離する場合がある点が被験者より指摘されている．

なお，人間の手動設計（黄色のマーク）においても，特にイテレーション 5 回目，9 回目，10 回目などで局所的な性能低下が観察される．これらは，人間が基本的には安定的な探索を指向しつつも，（三角マークが示す被験者のように）検証すべき仮説がある場合には大胆な変更も試みていることを示している．具体的には，5 回目に見られる低下は，Listing 6 に示すように，それまでサーバー数のみを考慮していた被験者が，システムの性能に影響を与えるもう一つの重要な変数（状態空間）として dimmer 値に着目し，その調整を試みるために既存のルール（サーバー数のみを考慮したルール）を意図的に破棄・変更したためである．

*4 詳細は GitHub リポジトリを参照：https://github.com/Uuusay1432/LLM-ARG

また，9 回目の低下は，Listing 7 に示すように，応答時間の閾値を RT_THRESHOLD から 0.1 * RT_THRESHOLD のように大幅に変更し，より積極的にサーバーを削減できるかという仮説検証（より大胆な閾値の探索）を行った結果と推察される．さらに，10 回目の低下は，サーバー調整と dimmer 調整のルールを統合する際に生じた実装ミスであり，複雑なロジックを構築する上での人為的な誤りである．これらの性能低下は，開発者の明確な仮説検証あるいは一時的なミスに基づいたものであり，LLM-ARG に多く見られる理解困難かつ突発的な変更とは異なる．

```
// 変更前のルール
if 条件(A) {
( dimmer の値は変更せず、起動するサーバーの数だけを変更する)
} else if 条件(B){
( dimmer の値は変更せず，起動するサーバーの数だけを変更する)
}

//変更後の性能が低下したルール
if 条件(A) {
( 起動するサーバーの数は変更せず，dimmer の値だけを変更する)
} else if 条件(B){
( 起動するサーバーの数は変更せず、dimmer の値だけを変更する)
}
```

Listing 6: 性能低下を引き起こした適応ルール変更例 1

```
// 変更前のルール
if (avgResponseTime > RT_THRESHOLD) {
( アクションA )
} else if (avgResponseTime < RT_THRESHOLD){
( アクションB )
}

//変更後の性能が低下したルール
if (avgResponseTime > 0.1*RT_THRESHOLD) {
( アクションA )
} else if (avgResponseTime <  0.3*RT_THRESHOLD){
( アクションB )
}
```

Listing 7: 性能低下を引き起こした適応ルール変更例 2

以上で述べた，LLM による過度に大胆な適応ルール変更に起因する不安定性の問題は，本質的には Exploration–exploitation dilemma であることが考えられる．具体的には，プロンプト上では「問題改善」と指示しているため，適応ルールに問題がなくても LLM は変更（Exploration）を試みてしまうことである．その結果，既存の性能の良いルールの構造を維持（Exploitation）せず，ルールを逆に改悪する傾向が生じうる．このような傾向は，自動設計の安定性を損なうだけでなく，人間が設計に対して持

© 2018 Information Processing Society of Japan

つ信頼感を低下させる要因ともなっている．将来研究として，システムの性能によって設計範囲を制約するメカニズムを取り組むべきである．これらの問題は他分野でも広く議論されており，強化学習ではソフトマックス方策によって探索確率を制御したり，進化計算では突然変異率や淘汰圧の調整によって過度な探索を抑制している．同様に，LLM の枠組みにおいても，性能に基づいて「改善幅を抑制する」仕組みを導入することが考えられる．具体的には，システムの効用が既に高い水準にある場合，LLM への指示を「既存構造を維持しつつ微調整を行う」ように切り替えるアプローチや，LLM の temperature パラメータを低く設定することで，LLM の応答をより決定論的にし，突飛なアイデアや大胆すぎる変更の生成を抑制するアプローチが考えられる．

また，本手法には汎用性を持つ部分とドメイン依存的な部分が存在する．汎用的な部分は，5 つのコンポーネントから成る反復的な改善フレームワークそのものである．この「運用履歴から問題を発見し，分析・改善・コード生成を繰り返す」というパイプラインは，特定のドメインに縛られないアプローチである．一方，ドメイン依存的な部分は，このフレームワークを特定のシステムに適用する際にカスタマイズが必要な箇所であり，その核心は Knowledge Base の構築にある．具体的には，対象システムごとに「観測・制御変数」「適応目標と効用関数」「ドメイン知識」を定義し，LLM に提供する必要がある．ただし，これらはソフトウェア開発においてシステムの仕様を定義する上で不可欠な情報であり，要求工学やドメイン分析といった既存の方法論を活用して，体系的に Knowledge Base を構築することが可能である．

4.5　妥当性への脅威

内的妥当性として 2 点挙げられる．まず，人間の被験者のドメイン知識や経験の差が結果に影響する可能性がある．また，実験では適応ルールが C++言語で表現されるため，C++言語に不慣れな被験者がルールの記述に苦慮する脅威も考えられる．さらに，本実験ではドメインをある程度熟知している少数の被験者に限定されているため，被験者の専門性と多様性が不足しており，結果の一般化が難しい一面もある．

外的妥当性としても 2 点挙げられる．まず，本研究はサーバー負荷分散の例題を用いた実験に基づくが，他のドメインアプリケーションへの適用可能性は十分に検証されていない．さらに，異なる適応目標や効用（例: ハード目標とソフト目標の組み合わせ）を設定した場合の有効性も未検証である．

5.　関連研究

5.1　適応ルール設計に関する研究

適応ルールの設計に関する研究は，「論理構文のみ表現されるルールの衝突検出」と「論理構文と数値の両方で表現されるルールの数値最適化」の 2 種類に分類できる．

前者の例として，[12] は大規模ホロニック機関におけるルール衝突検出に着目した．具体的には，衝突は同一サブ機関内の異なる役割間や複数サブ機関間で発生し，多ソート一階述語論理に基づく「イベント計算」で形式化・推論される．[7] はサイバー物理システム（CPS）向けルール言語「Rango」を提案した．具体的には，提案された構文（例: ノード，健康状態）は可読性を高め，手動設計の労力を削減するほか，双方向変換で従来バイナリ形式のルールとの互換性も確保した．[8] は負のパターン（例: コンポーネントの繰り返し故障）をアクション（例: コンポーネントの変更）に自動結びつけ，効用低下を防ぎ向上させるルール自動設計を提案した．[4] は IoT 向け ECA ルールの安全かつ効率的なデプロイに着目した．提案では，(1)「Heptagon/BZR」への変換で冗長性や衝突をモデルチェックし，(2) 優先順位付きルールから Java コントローラを自動生成する．

後者の例として，[23] は非機能要求を最大限満たすための適応ルールの設計に着目した．具体的には，運用履歴から重要性指標で重要な変数を抽出し，強化学習で非機能要求と変数の関係性を学習した上ルールを最適化している．さらに，[22] では変動する環境において，環境類似性指標を用いて，複数の既存適応ルールから，現在環境に一番適合するルールを選択する手法を提案した．[9] は Nomic ゲームを対象に，マルチエージェントシミュレーションでエージェント戦略（利己的，調和的，報復的，破壊的）が適応ルールの自己進化に与える影響を調査した．

これら既存研究では，状態空間は手動で行われており，本研究が対象とする状態空間の自動設計と最適化は行われていない．

5.2　大規模言語モデルを活用した最適化に関する研究

大規模言語モデル（LLM）を活用した最適化技術の研究は盛んである [3], [10], [20]．以下では，関連文献を 3 種類に分類し，簡潔に紹介する．まず，第 1 類は，自然言語などで問題文を LLM に与え，最適解を直接生成する手法である．この手法は，最適化勾配を直接定義することが困難な場面で主に適用される．本研究もこの類に属するが，巡回セールスマン問題やニューロエボリューションなど典型的な最適化問題を対象にする関連研究と異なり [14], [21]，本研究では自己適応システムのエンジニアリングにおける重要課題と指摘される状態空間設計 [5] を課題とし，その

© 2018 Information Processing Society of Japan

ために LLM-ARG を独自に設計している．次に，第 2 類は，LLM を最適化アルゴリズムの特定コンポーネントに活用する手法である．代表的な例として，進化計算アルゴリズムにおける交叉（crossover）や突然変異（mutation）を LLM に担わせることが挙げられる [2]．これにより，進化計算がテキストなどより複雑なデータ表現を扱えるようになる．最後に，第 3 類は，LLM を用いて適切な最適化アルゴリズムを選択・生成する手法である．この場合，LLM をアルゴリズムの内部コンポーネントの調整に利用するのでなく，最適化アルゴリズムそのものを選択と設計するために用いられている [19]．

6. おわりに

本論文では，適応ルールの生成と改善に LLM を活用する手法として LLM-ARG を提案した．本手法は，LLM の高い推論能力を利用し，システムの運用データを基に適応ルールの分析と改良を繰り返す．これにより，実際の運用環境に対してそれぞれ品質の高い適応ルール設計を実現する．サーバ負荷分散の例題での実験を通じて，LLM による適応ルール生成は従来人手での設計に比べコストを削減しつつ，手動設計と同等な性能の実現が確認できた．

今後の研究として，以下の 3 つの方向性を検討する．第一に，LLM による状態空間の最適化と既存の最適化アルゴリズムによる数値最適化を組み合わせることで，より高性能な適応ルールの構築が期待できる [15]．第二に，フレームワークとプロンプトのさらなる改良により，LLM による適応ルール改善プロセスの安定性を高めることである．例えば，適応ルールの性能次第 LLM の指示を変えることで，良い性能時に LLM が既存の設計方針を大きく逸脱するのを防ぐことが考えられる．第三に，想定外のリクエスト率変化にも対応可能になるよう，適応ルールのロバストネスなどの性質もルール設計に考慮することである．

参考文献

[1] Mathieu Acher, Philippe Collet, Franck Fleurey, Philippe Lahire, Sabine Moisan, and Jean-Paul Rigault. Modeling context and dynamic adaptations with feature models. *Proceedings of the 4th International Workshop Models@run.time*, Vol. 509, , 10 2009.

[2] Jinyu Cai, Jialong Li, Mingyue Zhang, Munan Li, Chen-Shu Wang, and Kenji Tei. Language evolution for evading social media regulation via llm-based multi-agent simulation. In *2024 IEEE Congress on Evolutionary Computation (CEC)*, pp. 1–10, 2024.

[3] Jinyu Cai, Jinglue Xu, Jialong Li, Takuto Yamauchi, Hitoshi Iba, and Kenji Tei. Exploring the improvement of evolutionary computation via large language models. In *Proceedings of the Genetic and Evolutionary Computation Conference Companion (GECCO)*, p. 83–84, 2024.

[4] Julio Cano, Eric Rutten, Gwenaël Delaval, Yazid Benazzouz, and Levent Gurgen. Eca rules for iot environment: A case study in safe design. In *2014 IEEE Eighth International Conference on Self-Adaptive and Self-Organizing Systems Workshops*, pp. 116–121, 2014.

[5] Betty H. C. Cheng, Rogério de Lemos, Holger Giese, Paola Inverardi, Jeff Magee, Jesper Andersson, Basil Becker, Nelly Bencomo, Yuriy Brun, Bojan Cukic, Giovanna Di Marzo Serugendo, Schahram Dustdar, Anthony Finkelstein, Cristina Gacek, Kurt Geihs, Vincenzo Grassi, Gabor Karsai, Holger M. Kienle, Jeff Kramer, Marin Litoiu, Sam Malek, Raffaela Mirandola, Hausi A. Müller, Sooyong Park, Mary Shaw, Matthias Tichy, Massimo Tivoli, Danny Weyns, and Jon Whittle. *Software Engineering for Self-Adaptive Systems: A Research Roadmap*, pp. 1–26. Springer Berlin Heidelberg, Berlin, Heidelberg, 2009.

[6] Shang-Wen Cheng, David Garlan, and Bradley Schmerl. Evaluating the effectiveness of the rainbow self-adaptive system. In *2009 ICSE Workshop on Software Engineering for Adaptive and Self-Managing Systems*, pp. 132–141, 2009.

[7] Melanie Feist, Martin Breitbach, Heiko Trötsch, Christian Becker, and Christian Krupitzer. Rango: An intuitive rule language for learning classifier systems in cyber-physical systems. In *2022 IEEE International Conference on Autonomic Computing and Self-Organizing Systems (ACSOS)*, pp. 31–40, 2022.

[8] Sona Ghahremani, Holger Giese, and Thomas Vogel. Towards linking adaptation rules to the utility function for dynamic architectures. In *2016 IEEE 10th International Conference on Self-Adaptive and Self-Organizing Systems (SASO)*, pp. 142–143, 2016.

[9] Stuart Holland, Jeremy Pitt, David Sanderson, and Dídac Busquets. Reasoning and reflection in the game of nomic: Self-organising self-aware agents with mutable rule-sets. In *2013 IEEE 7th International Conference on Self-Adaptation and Self-Organizing Systems Workshops (SASO)*, pp. 101–106, 2013.

[10] Sen Huang, Kaixiang Yang, Sheng Qi, and Rui Wang. When large language model meets optimization. *Swarm and Evolutionary Computation*, Vol. 90, p. 101663, 2024.

[11] Yusei Ishimizu, Jialong Li, Jinglue Xu, Jinyu Cai, Hitoshi Iba, and Kenji Tei. Automatic adaptation rule optimization via large language models. In *2024 IEEE International Conference on Autonomic Computing and Self-Organizing Systems Companion (ACSOS-C)*, pp. 180–181, 2024.

[12] Jie Jiang, Jeremy Pitt, and Ada Diaconescu. Rule conflicts in holonic institutions. In *2015 IEEE International Conference on Self-Adaptive and Self-Organizing Systems Workshops*, pp. 49–54, 2015.

[13] Ivan Lanese, Antonio Bucchiarone, and Fabrizio Montesi. A framework for rule-based dynamic adaptation. In *Proceedings of the 5th International Conference on Trustworthly Global Computing (TGC)*, p. 284–300, Berlin, Heidelberg, 2010. Springer-Verlag.

[14] Robert Lange, Yingtao Tian, and Yujin Tang. Large language models as evolution strategies. In *Genetic and Evolutionary Computation Conference (GECCO) Companion*, p. 579–582, 2024.

[15] Jialong Li, Mingyue Zhang, Nianyu Li, Danny Weyns, Zhi Jin, and Kenji Tei. Generative ai for self-adaptive systems: State of the art and research roadmap. *ACM Trans. Auton. Adapt. Syst.*, Vol. 19, No. 3, September 2024.

[16] Gabriel A. Moreno, Bradley Schmerl, and David Garlan. Swim: An exemplar for evaluation and comparison of

© 2018 Information Processing Society of Japan

self-adaptation approaches for web applications. In *2018 IEEE/ACM 13th International Symposium on Software Engineering for Adaptive and Self-Managing Systems (SEAMS@ICSE)*, pp. 137–143, 2018.

[17] Ashutosh Pandey, Gabriel A. Moreno, Javier Cámara, and David Garlan. Hybrid planning for decision making in self-adaptive systems. In *2016 IEEE 10th International Conference on Self-Adaptive and Self-Organizing Systems (SASO)*, pp. 130–139, 2016.

[18] ClarkNet Project. Clarknet-http dataset, 1995. Accessed: 2025-03-13.

[19] Xingyu Wu, Yan Zhong, Jibin Wu, Bingbing Jiang, and Kay Chen Tan. Large language model-enhanced algorithm selection: towards comprehensive algorithm representation. In *Thirty-Third International Joint Conference on Artificial Intelligence (IJCAI)*, 2024.

[20] Jinglue Xu, Jialong Li, Zhen Liu, NAV Suryanarayanan, Guoyuan Zhou, JIA GUO, Hitoshi Iba, and Kenji Tei. Large language models synergize with automated machine learning. *Transactions on Machine Learning Research (TMLR)*, 2024.

[21] Chengrun Yang, Xuezhi Wang, Yifeng Lu, Hanxiao Liu, Quoc V Le, Denny Zhou, and Xinyun Chen. Large language models as optimizers. In *The Twelfth International Conference on Learning Representations (ICLR)*, 2024.

[22] Tianqi Zhao. The generation and evolution of adaptation rules in requirements driven self-adaptive systems. In *2016 IEEE 24th International Requirements Engineering Conference (RE)*, pp. 456–461, 2016.

[23] Tianqi Zhao, Tao Zan, Haiyan Zhao, Zhenjiang Hu, and Zhi Jin. Integrating goal model into rule-based adaptation. In *2016 23rd Asia-Pacific Software Engineering Conference (APSEC)*, pp. 289–296, 2016.

謝辞

本研究で行ったユーザ実験は，実験実施した機関（東京科学大学および早稲田大学）の規定上倫理審査の対象範囲外である．被験者の意思確認のため，実験前に実験の目的およびデータの保存・開示方法を説明し，同意を得た．本研究は，JSPS 科研費（23K28064，25K15290）による一部支援を受けたものである．

付録

3 章で説明した Problem Identifier，Rule Optimizer，及び Rule Generator のプロンプトの構造を示す．

```
DomainKnowledge: {domainKnow_str}
AdaptationGoal: {goal_str}
ViriableInfo: {variable_str}
ExecutionHistory: {history_str}
TaskDescription:{"Please select areas of your
    implementing code where results show that the
    utilities can be improved, server can be reduced
    , or dimmer can be higher when responseTime is
    low, during which time period and what is the
    cause of the low utility at that period?"}
```

Listing 8: Problem Identifier のステップ 2 のプロンプト構造：運用履歴における改善箇所特定

```
DomainKnowledge: {domainKnow_str}
AdaptationGoal: {goal_str}
ViriableInfo: {variable_str}
ExecutionHistory: {history_str}
HistoryProblem: {historyproblem_str} // 補足説明
TaskDescription:{
"What is one of the biggest problems in the design of
    the rules for the evaluation method that caused
    this? Which part of the code would you identify
    as the cause of the problem?"}
SouceCode: {code_str}
```

Listing 9: Problem Identifier のステップ 2 のプロンプト構造：適応ルールにおける問題箇所特定

```
DomainKnowledge: {domainKnow_str}
AdaptationGoal: {goal_str}
VariableInfo: {variable_str}
ExecutionHistory: {history_str}
HistoryProblem: {historyproblem_str}
CodeProblem: {codeproblem_str}
TaskDescription:{"Please explain what is your plan
    that can be achieved for improvement while
    meeting the implementation constraints in your
    identified area. Why do you consider this
    improvement?"}
SouceCode: {code_str}
```

Listing 10: Rule Refiner のプロンプト構造

```
DomainKnowledge: {domainKnow_str}
AdaptationGoal: {goal_str}
VariableInfo: {variable_str}
ExecutionHistory: {history_str}
HistoryProblem: {historyproblem_str}
CodeProblem: {codeproblem_str}
ImprovementPlan: {improvementplan_str}
TaskDescription:{"Please make new rules which refer
    to what conditions should increase or decrease
    the number of servers or change the dimmer value
    , and implement new evaluate method in
    MyAdaptationManager.cpp to aim to maximize the
    sum of utilities calculated by
    periodUtilityICAC2016 over the entire period of
    the simulation with reference to what you have
    analyzed until now and the following. The past 3
     problems and improvements of implementation, in
     order from newest to oldest"}
SouceCode: {code_str}
```

Listing 11: Rule Generator のプロンプト構造

石水 優成　（学生会員）

2025 年東京科学大学情報理工学院情報工学系学士課程を修了.2025 年より同大学修士課程に所属し, 自己適応システム，大規模言語モデルに関する研究に従事.

© 2018 Information Processing Society of Japan

許経略

2025 年東京大学大学院情報理工学系研究科博士課程修了. 現在, Sakana AI にて研究員として勤務．LLM エージェント，進化計算，および AutoML の研究に従事.

山内 拓人　（正会員）

2024 年に早稲田大学理工学術院基幹理工学研究科博士課程修了．同大学助手を経て，2024 年より早稲田大学講師．東京科学大学特別研究員を兼任．博士 (工学) (早稲田大学)．自己適応システム，要求工学の研究に従事.

蔡 金雨

2023 年早稲田大学理工学術院基幹理工学研究科情報理工・情報通信専攻修士課程を修了．2023 年より同大学博士後期課程に所属し，自律エージェントや大規模言語モデルに関する研究に従事.

伊庭 斉志

1990 年東京大学大学院工学系研究科情報工学専攻博士課程修了．工学博士．同年電子技術総合研究所入所．1996 年から 1997 年までスタンフォード大学客員研究員．1998 年から東京大学大学院工学系研究科電子情報工学専攻助教授．2004 年から同教授．2011 年から同大学大学院情報理工学系研究科電子情報学専攻教授．進化システム，人工生命及び人工知能基礎の研究に従事.

李 家隆　（正会員）

2025 年早稲田大学基幹理工学研究科情報理工・情報通信専攻博士課程修了．同大学助手経て 2025 年より早稲田大学講師，東京科学大学特別研究員 (兼任)，現在に至る．自己適応システム，要求工学の研究に従事

鄭 顕志　（正会員）

2008 年早稲田大学大学院理工学研究科博士課程修了．同大学助手，助教，国立情報学研究所助教，准教授，早稲田大学研究院准教授/主任研究員を経て 2024 年より東京科学大学准教授．現在に至る．現在，国立情報学研究所 GRACE センター, 早稲田大学特任准教授を兼任．博士 (工学) (早稲田大学)．自己適応システム，ソフトウェアアーキテクチャ，モデル駆動工学の研究に従事．本会シニア会員.

© 2018 Information Processing Society of Japan